

Figure 4: XMPP architecture

like at-most-once, at-least-once and exactly-once delivery. The AMQP protocol consists of a set of components that route and store messages within a broker service, with a set of rules for wiring the components together. The AMQP protocol enables client applications to talk to the broker and interact with the AMQP model. This model has the following three components, which are connected into processing chains in the server to create the desired functionality.

- **Exchange:** Receives messages from publisher based applications and routes them to 'message queues'.
- **Message queue:** Stores messages until they can be safely processed by the consuming client application.
- **Binding:** States the relationship between the message queue and the exchange.

Data Distribution Service (DDS): This IoT protocol for real-time machine-to-machine communication was developed by the Object Management Group (OMG). It enables scalable, real-time, dependable, high-performance and interoperable data exchange via the publish-subscribe methodology. As compared to MQTT and CoAP IoT protocols, DDS makes use of brokerless architecture and of multicasting to bring high quality QoS to applications. DDS can be deployed in platforms ranging from low-footprint devices to the cloud, and supports efficient bandwidth usage as well as the agile orchestration of system components.

The DDS protocol has two main layers: Data Centric Publish-Subscribe (DCPS) and Data-Local Reconstruction Layer (DLRL). DCPS performs the task of delivering the information to subscribers, and the DLRL layer provides an interface to DCPS functionalities, enabling the sharing of distributed data among IoT enabled objects.

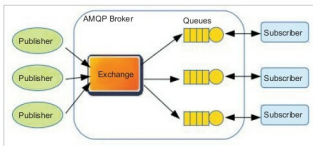


Figure 5: AMQP architecture

STOMP (Simple Text Oriented Messaging Protocol):

This text based protocol was developed to work with message-oriented middleware. It provides an interoperable wire format that enables STOMP clients to communicate with any STOMP message broker to enable easy and widespread messaging interoperability among many languages, platforms and brokers. Like AMQP, STOMP provides the message header with properties and a frame body.

STOMP does not, however, deal in queues and topics—it uses a SEND semantic with a ‘destination’ string. The broker must map it onto something that it understands internally, such as a topic, queue, or exchange. Consumers then SUBSCRIBE to those destinations. Since those destinations are not mandated in the specifications, different brokers may support different flavours of the destination. So, it’s not always straightforward to port code between brokers.

However, STOMP is simple and lightweight (although somewhat verbose on the wire), with a wide range of language bindings. It also provides some transactional semantics. One of the most interesting examples is with RabbitMQ Web Stomp, which allows you to expose messaging in a browser through Websockets.

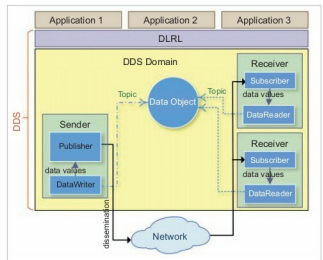


Figure 6: DDS protocol architecture

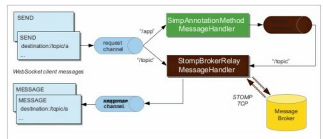


Figure 7: STOMP architecture

Continued on Page 90...